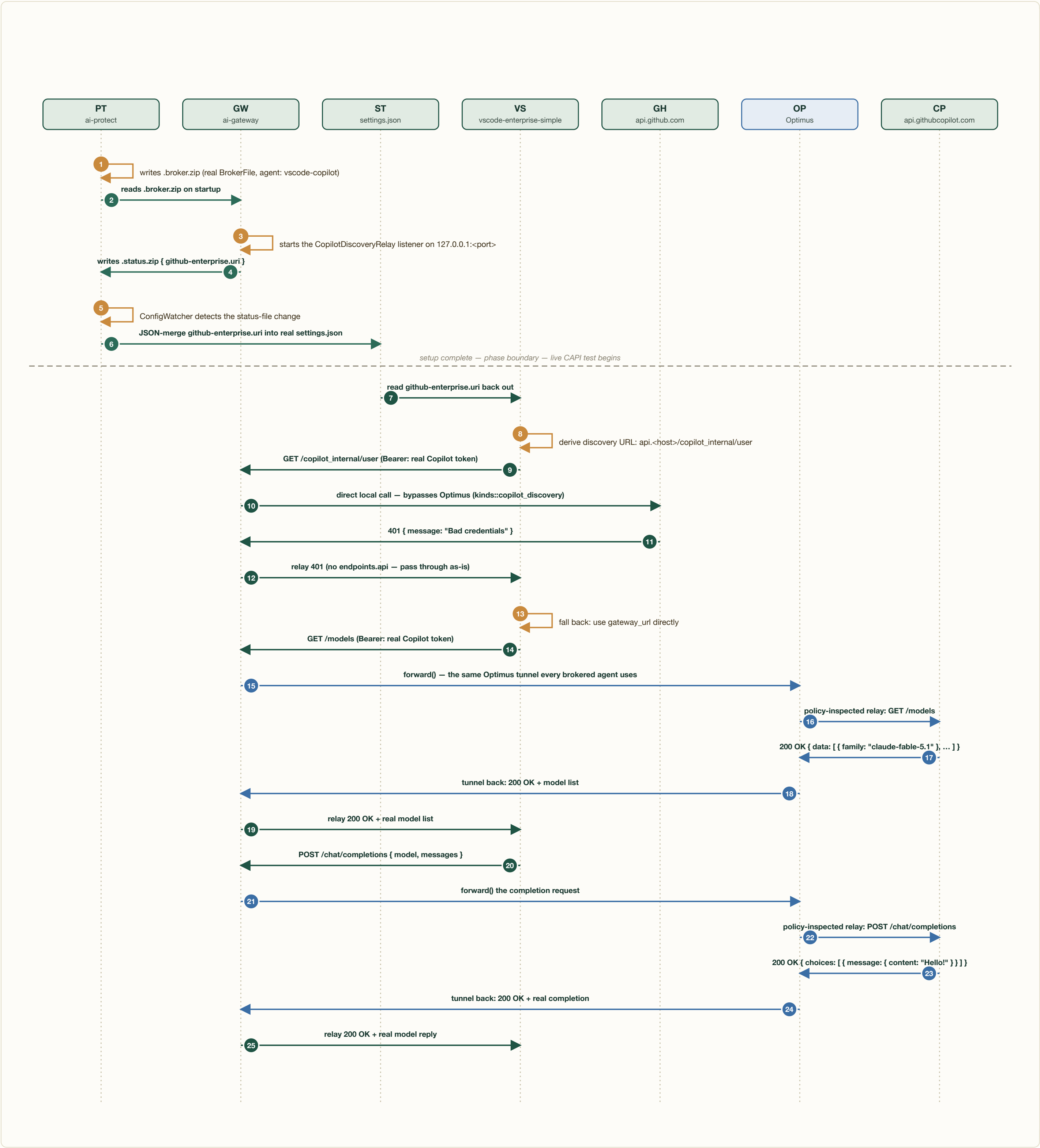


Planned real-VM test — the real ai-protect / ai-gateway, Optimus in the loop NOT YET RUN

What the throwaway Windows VM test is expected to show once run: the real, shipped ai-protect/ai-gateway (commit a47e68f0), not the standalone -simple precursor tools — discovery still answered directly (bypassing Optimus, by design), but /models and /chat/completions now genuinely tunnel through Optimus like every other brokered agent's LLM traffic.

- setup phase (local file wiring)
- direct call (discovery only)
- Optimus-tunneled call
- self action



Step-by-step

- 1

ai-protect writes .broker.zip PT → (self)
The real `BrokerFile` the shipped daemon writes once it has enrolled and detected VS Code + the Copilot Chat extension on the VM — naming the real `vscode-copilot` agent (commit `a47e68f0`), not a standalone stand-in.
- 2

ai-gateway reads it on startup PT → GW
The real listener process, confirming it should serve the `vscode-copilot` agent before binding anything.
- 3

ai-gateway starts the CopilotDiscoveryRelay listener GW (self)
`listener::kinds::copilot_discovery` — the real shipped module. Its `ListenerTransport` is derived automatically from the agent's `LlmRouting::CopilotDiscoveryRelay` variant, so this can't drift from step 1's registration.
- 4

Writes .status.zip GW → PT
The real `StatusFile` / `ConfigDelta`, naming the resolved `github-enterprise.uri` for ai-protect to render into config.
- 5

ai-protect's ConfigWatcher detects the change PT (self)
The same file-watch path every other agent's config delta goes through — nothing special-cased for this agent.
- 6

JSON-merge into the real settings.json PT → ST
Rendered via the existing `Llm::ProxyPath` shape (already proven for Kiro's `http.proxy`) — no new render code needed for this agent.
- 7

vscode-enterprise-simple reads github-enterprise.uri back out ST → VS
Standing in for a real, enterprise-flagged VS Code Copilot Chat install — confirms the merge in step 6 actually landed on the VM.
- 8

Derives the discovery URL VS (self)
VS Code's own exact formula, traced from `defaultAccount.ts`: prepend `api.` onto the configured hostname, unconditionally.
- 9

GET /copilot_internal/user VS → GW
Sent with a real Copilot bearer token — minted via `ai-gateway/scripts/get_copilot_token.sh`'s OAuth device flow, copied onto the VM as `token.txt` — and the real client headers.
- 10

Direct local call to the real api.github.com GW → GH
This is the one leg the real listener genuinely makes itself, bypassing Optimus entirely — it's account-entitlement metadata, not agent traffic that needs policy inspection. Matches the design in §4 of the write-up exactly, not a simplification.
- 11

401 Bad credentials GH → GW
Expected — same known token-type mismatch as the original live test (this is the Copilot-specific token, not the raw GitHub OAuth token discovery wants). Not a failure of the relay. **Caveat carried over from the original test: this VM run is also not expected to see a real, successful discovery response.** A genuine `200` with `endpoints.api` actually being rewritten would need the raw GitHub OAuth token instead — a different flow neither test tool acquires — so even here the rewrite path stays confirmed by source-reading, not by a live observation.
- 12

Relay the 401 back untouched GW → VS
No `endpoints.api` field in a 401 to rewrite, so it passes straight through.
- 13

Client falls back to the gateway URL directly VS (self)
Exactly what it would do if discovery had succeeded and named this same gateway as the CAPI host.
- 14

GET /models VS → GW
A real, authenticated listing call sent to the real listener.
- 15

ai-gateway calls forward() GW → OP
The exact same Optimus-tunnel primitive every other `BaseUrl` agent in this product already uses — **not** a direct outbound call. This is the real listener's actual path, unlike the original mechanism-proof test.
- 16

Optimus relays GET /models, policy applied OP → CP
Optimus decides whether this request is allowed before it ever reaches GitHub's servers — the enforcement point every other brokered agent's LLM traffic already passes through.
- 17

200 OK, a real model list CP → OP
Genuine capability metadata from the real `api.githubcopilot.com`, returned to Optimus first.
- 18

Optimus tunnels the response back OP → GW
Back to ai-gateway — the response leg of the same tunnel, not a second, independent call.
- 19

Relay 200 OK + model list to the client GW → VS
Round-trips intact through both the tunnel and the local listener.
- 20

POST /chat/completions VS → GW
The real inference request — same payload shape already validated directly against real CAPI.
- 21

ai-gateway calls forward() again GW → OP
Same tunnel primitive as step 15, now carrying a generation request instead of a listing.
- 22

Optimus relays POST /chat/completions, policy applied OP → CP
The actual model-generation call is inspected by policy exactly like every other agent's completion traffic — this is the property the hybrid design in §4 exists to preserve.
- 23

200 OK, a genuine reply from a real model CP → OP
Expected to round-trip the same way the original mechanism-proof test did (a real model replying to the test prompt with real token accounting) — now through Optimus instead of a direct call.
- 24

Optimus tunnels the response back OP → GW
Back to ai-gateway, completing the second tunnel round-trip.
- 25

Relay 200 OK + real model reply to the client GW → VS
The full, real, shipped path — discovery direct, everything else through Optimus — proven end to end, on a disposable VM instead of a standalone test harness.